LA BOEM QUANDO LE CHIEDI DI MISURARE I RISULTATI DELL'ESPERIMENTO PRIMA DEL SECONDO



~~APPELLO~~ RISULTATO

↙ liceo

30/9

- Compilare form
- Codice e-learning: 13683
- Inizio meccanica 20/10
- Non più di 3 assenze per prova valutata
- Compilatore C gcc] Da installare
- Python 3.x]
- Stanza 607 4° piano fermi: Boeri merc: 11-12
- Stanza 116 1° piano: Spagnolo, Martedì 12-13, se fuori orario contattare per email.
- Installare Linux sul computer di casa
- Libro di: Testo + dispense [Programmazione scientifica L. Barone"] nell'esonero si è singoli, Tutte le tre prove
- Lab Ponte corvo: Tiburtina 205
- Esame:
  - Prova pratica: 3 ore
  - Bonus nella prova valutata se in sessione invernale:
    - 18-23: 1
    - 24-25: 2
    - 26-27: 3
    - 28-29: 4
    - 30/30 e lode esonera prova finale.
  - Bonus con risoluzione + spiegazione esercizi assegnati:

Argomenti corso:

- Parte 1: Come funziona un computer
- Parte 2: Elementi di base del C
- Parte 3: Elementi avanzati dello C
- Parte 4: Applicazioni e algoritmi + Python.

1/10

Principi di funzionamento di un computer.

Sistema di numerazione: insieme di regole con cui i numeri vengono rappresentati:

decimale: 0, 1, ..., 8, 9, sistema posizionale → $(123)_{10} \neq (231)_{10}$

$10^2 \; | \; 10^1 \; | \; 10^0$ → $1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0 = \sum_{k=0}^{N_c-1} c_k \, b^k$

(N_c: numero cifre; c_k: cifra; b^k: peso; b: base; k: esima cifra)

Cifre significative → da l'ordine di grandezza, in 123, è $1 \cdot 10^2$, quindi il range è da 101 a 199

Nella somma decimale, $A = (123)_{10}$, $B = (456)_{10}$, $C = A + B$

$$C = \sum_{k=0}^{N_c-1} \gamma_k \cdot b^k = \sum_{k=0}^{N_c-1} (\alpha_k + \beta_k) \cdot b^k$$

$$(101)_2 = (5)_{10} = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 = (1+4)_{10} = (5)_{10}$$

128	64	32	16	8	4	2	1
0	0	0	0	0	1	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

→ max con 8 cifre $255 = 2^8 - 1$

$N_{max} = b^{N_c} - 1$

Conversione dec → binario: metodo dei resti:

$n = (43)_{10} = (\;?\;)_n$

43	2	: 1	↑ dx
21	2	: 1	
10	2	: 0	$(101011)_2 = (43)_{10}$
5	2	: 1	
2	2	: 0	
1	2	: 1	sx
0			

Algoritmo: $N_i = b \cdot N_{i+1} + r_i$ (b: base; r_i: resto), $r_i \in \mathbb{Z} \mid r_i \in [0; b[$

$(32)_{10} = (\;?\;)_3 = (1012)_3$

N_0	32	3 : 2	r_0
N_1	10	3 : 1	r_1
N_2	3	3 : 0	r_2
N_3	1	3 : 1	r_3
	0		

$32 = 10 \cdot 3 + 2$

$N_0 = N_1 \cdot b + r_0$

$10 = 3 \cdot 3 + 1$

$N_1 = N_2 \cdot b + r_1$

$3 = 1 \cdot 3 + 0$

$N_2 = N_3 \cdot b + r_2$

$1 = 0 \cdot 3 + 1$

$N_3 = N_4 \cdot b + r_3$

A/10

Ci mettono 32/64 bit.

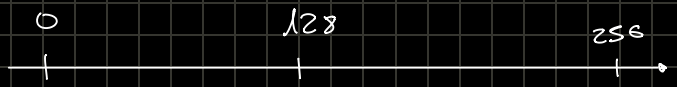
Oltre il binario, si usa anche l'Hex:

$b = 16 = 2^4$

0, ..., 9, A, B, C, D, E, F

+12
−12
} → si rappresenta usando l'ultima cifra a sinistra, se 1 → +, 0 → −
segno

Usando $n_c = 7$, e quindi il $n_{max} = 127$ e $n_{min} = -127$ → 255 numeri





$N_{INT} = N_S + 128$ → notazione in eccesso (non molto usata)

Prec. m, per la notazione in complemento, $(m) + (K) = (b^{n_c})$ → numero cifre
m → num da rappresentare, K → complemento, b → base

$b = 10$
$n_c = 3$
} → $N_{max} = 999$

$m = 32$, $b = 10$, $n_c = 3$ ⇒ $K = 968$

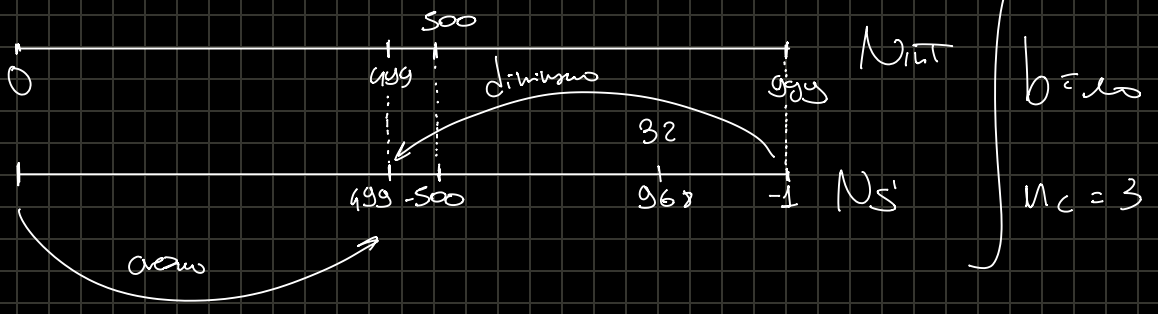
Study:

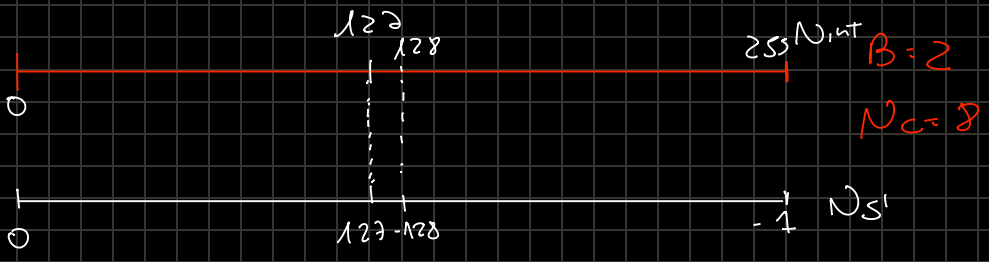
$$\begin{array}{r} 999 \\ -\ 32 \\ \hline 967 \end{array} + 1 = 968$$

$$K = b^{n_c} - m = (b^{n_c} - 1) - m + 1$$

In base 10:

da 0 a 499, li leggo letteralmente, non pos
da 500 a 999, li leggo in complemento, num negativi





Sottrazioni in complemento.

$32 \Leftrightarrow 968$
"-32"

$$\begin{array}{r} 127 - \\ 32 \\ \hline 95 \end{array} \qquad \begin{array}{r} 127 + \\ 968 \\ \hline 1\,095 \end{array}$$

Prendo solo le prime N_c cifre

In Base 2:

$b = 2,\ n_c = 8,\ N_{max} = 256,\ m = (32)_{10} \Rightarrow (00100000)_2$

$k = (224)_{10} \Rightarrow (11011111)_2 + 1 = (11100000)_2 = (224)_{10}$

$(127)_{10} = (01111111)_2$

$$\begin{array}{r|l} 127 - & 01111111 + \\ 32 & 11100000 \\ \hline 95 & 1\,01011111 \end{array}$$

solo le prime n_c cifre

$(01011111)_2 = (95)_{10}$

Come rappresentare la virgola, (o punto)

$(N)_b = \sum_{k=0}^{nc-1} b^k \cdot c_k$, solo se interi, dato che $k >$

$12.371 = 1 \cdot 10^1 + 2 \cdot 10^0 + 3 \cdot 10^{-1} + 7 \cdot 10^{-2} + 1 \cdot 10^{-3}$

$N_I = 2$ con: $\sum_{k=N_D}^{N_I-1} c_k \cdot b^k$, generalizzazione di $(N)_b$ con virgola.

$N_D = 3$

$(10.011)_2 = (2.375)$

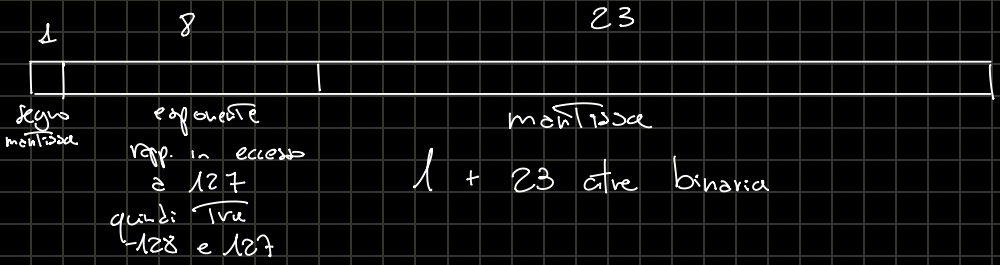
$2^1\ 2^0\ 2^{-1}\ 2^{-2}\ 2^{-3}$

Virgola Mobile: rappresentazione normale

$12.371 = [1.2371 \cdot 10^1] = 123.71 \cdot 10^{-1}$

(Mantissa, parte prima della virgola) $\cdot 10^{exp}$

$12\,000\,000\,000 = 12 \cdot 10^{10}$



"Float", 32 bit detti a virgola mobile.

"Integer", senza di potenze di due.

Operatori logici:

AND:

OR:

NOT:

T	F
1	0

(oggi piove) $\wedge$ (ho l'ombrello)

AND

&&

AND	P1	P2
1	1	1
0	1	0
0	0	1
0	0	0

OR	P1	P2
1	1	1
1	1	0
1	0	1
0	0	0

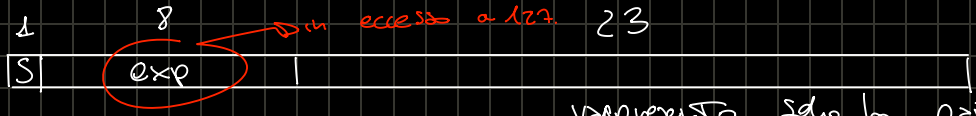
NOT	P1
1	0
0	1

Codice ASCII, usa 256, e mappa ogni lettera ad numero.

In C:

Float 32 bit

Double 64 bit



$$(N)_{\text{FLOATING POINT}} = M \cdot (b)^{exp}$$

In binario: $1.xxxx... \cdot 2^{exp}$

$$(-1)^{S} \cdot (1.M)\, 2^{(exp-127)}$$

$$N_{max} \simeq 2^{128} \cdot 2 \simeq 10^{38}$$

$$N_{min} \simeq 2^{127}\left(1+10^{-23}\right) \simeq 2^{-127} \simeq 10^{-38}$$

Exp = max, $M = 0 \rightarrow N \in \mathbb{N}$, ∞

, $M \neq 0 \rightarrow$ massimo

Conversione da $(N)_{10} \rightarrow (N)_{\text{FLOATING POINT}}$

$(10.75)_{10} \longrightarrow$

1) Converto in binario

$[1010.11]$

2) Riduco in notazione F.P. NORMALE

$1010.11 \rightarrow 1.01011 \cdot 2^{3}$

S | 1 0 0 0 0 0 1 0 | 0 1 0 1 1 0 0 0 | 0 0 0 0

$(-1)^{S} \cdot (1.M) \cdot 2^{(exp-127)} \rightarrow S = 0$

EXP = 130 = 10000010

M = 01011

Non tutti i numeri decimali hanno rappresentazione come somma di potenze di due. Tipo 0.1

Parti del PC

RAM = Random Access Memory, organizzata in registri, con indirizzi univoci.

CPU = Control Processing Unit

↳ Tempo di clock = $\frac{1}{\text{frequenza di clock}}$ → num operazioni al secondo, GHz ~ 10^9 operazioni

Dispositivi di I/O:

I: Tastiera, mouse, microfono, video camera, Touch screen, Schede di acquisizione

O: Monitor, Speaker, Stampante

- Memorie permanenti:
  - HDD/SDD

- Ram della CPU: memoria cache

La CPU fa:

- Spostamento di DATI: RAM → CPU
  CPU → RAM
- Calcolo (op. algebriche / logiche)
  + − · / or and not
- Gestione di flusso: Iterazione (for)
  condizionale (if / Then)

14/10

Macchina di Turing (1936) [ASTRATTA]

- prima formalizzazione rigorosa di un algoritmo.

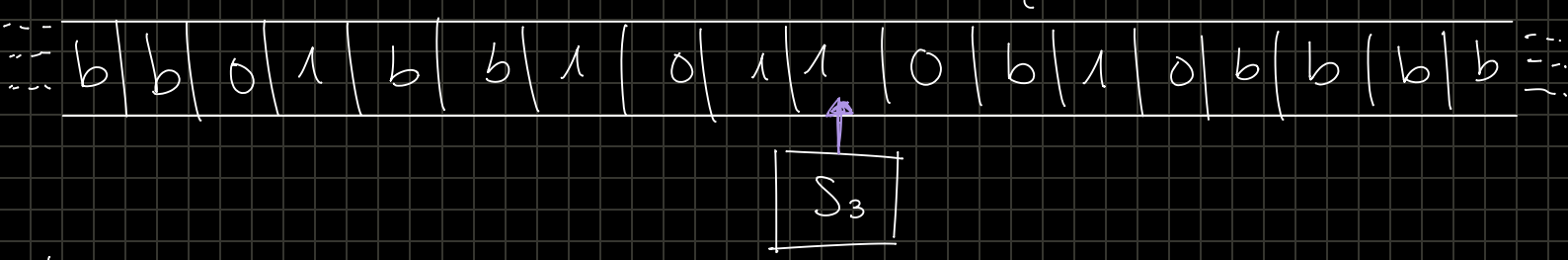
Algoritmo: sequenza di istruzioni ordinate per risolvere un problema noto

ha Tre aspetti fondamentali:

- memoria
- unità di controllo
- unità di lettura/scrittura

Funzionamento:

preso un nastro infinito, diviso in celle, $\{0,1\}$



dove posso scrivere uno Tra gli elementi di un alfabeto

$\{a_1, \dots, a_k\} + b(\text{blank: "vuoto"})$

Finito di celle si trova in un certo stato $(\{a_1, \dots, a_k\})$ e le altre sono vuote

Unità di controllo, ha un suo configurazione, si Tra le configurazioni possibili: $\{s_1, \dots, s_e\}$ + H (halting) → stato che identifica la fine del processo

3) Unità di lettura / Scrittura

si muove lungo il nastro a sx/dx di una cella.

La MdT è in grado di risolvere TUTTI i possibili problemi risolvibili con algoritmi, secondo la Tesi di Church-Turing.

Funzione con un programma, avremo le regole.

- Istruzione / Step $(s, a) \to (\bar{s}, \bar{a}, d)$

1: Come cambia lo stato dell'unità di controllo

$$s \longrightarrow \bar{s}$$

2: Come cambia il contenuto della cella

$$a \longrightarrow \bar{a}$$

3: Spostamento della Testina

Calcola di 2+3 con la macchina di Turing

Alfabeto unario: $\{1\} + b$

$1_{10} \to 1$

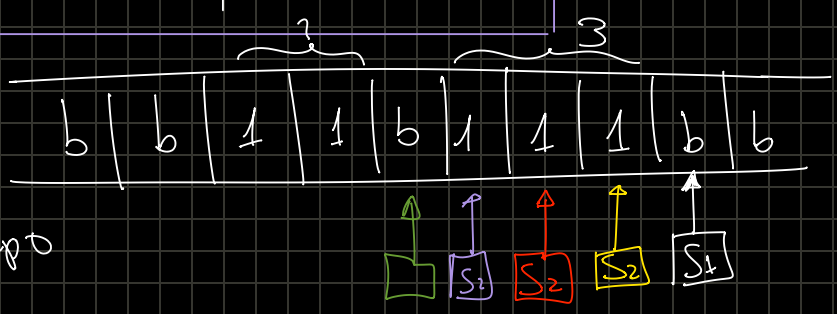
$2_{10} \to 11$

$\vdots$

Stati: $\{s_1, s_2, s_3, s_4, H\}$

Programma $(s, a) \to (\bar{s}, \bar{a}, d)$

s	a	s	a	d
s_1	b	s_2	b	e
s_2	b	s_3	b	e
s_2	1	s_2	1	e
s_3	b	H	b	o
s_3	1	s_4	b	r
s_4	b	s_2	1	e



Step 0

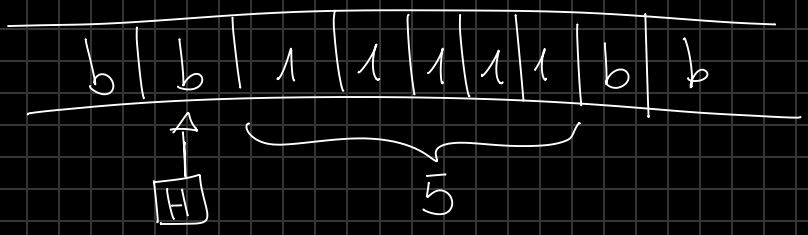
Step 1:

Step 2:

Step 3:

Step 4:

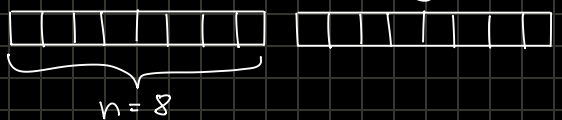
$\vdots$



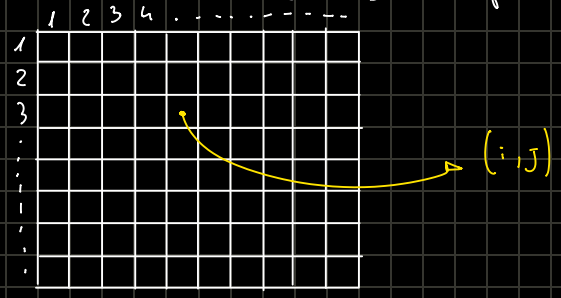
Calcolatore moderno:

Elementi:

- Control processing unit (CPU), esegue una sequenza ordinata di istruzioni su dati in Memoria
- Memoria: dispositivo fisico dove salvare i dati, serie ordinata di celle, su cui immagazzinare bit.
  - RAM: Random Access Memory



hanno un certo indirizzo: ad esempio



- Registri della CPU:

La CPU fa 3 operazioni:

1) Spostamenti dati:
   - RAM → CPU
   - CPU → RAM
   - CPU → CPU
   - RAM → RAM
2) Operazioni elementari: (logiche ed algebriche)
3) Controllo di flusso

Viene caratterizzata dalla frequenza di clock, scandisce le operazioni della CPU

L_0, m^N bit — N: dipende dall'architettura; m: num intero

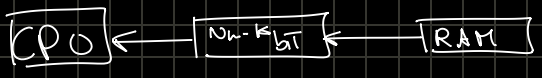
1) $T_{clock} = 0$



$[0,0,0,1,0,1,1,0]$ Op Code, ovvero una istruzione

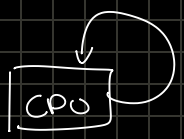
2) $N_m - k$ bit successivi:

$T_{clock} = 1$



3) T_{clock} successivi

esegue l'istruzione



esempi di istruzioni:

$[0,0,0,0,0,0,0,0]$: legge un certo indirizzo di memoria e salvalo nel registro.

Il programma è una sequenza di bit che scrivo nella memoria secondo delle regole.

APPROFONDIMENTO ASSEMBLY, BOOOSS!!!!

Linguaggi di programmazione

- Il livello macchina, il più basso livello di astrazione
- Assembly: livello successivo rispetto all'astrazione, richiede l'assemblatore, che traduce le istruzioni in bit

- Linguaggi compilati: dopo aver scritto le istruzioni il compilatore le converte in bit per il codice macchina, della macchina su cui è stato compilato, quindi lo eseguo è più veloce è già stato tradotto
- Linguaggi interpretati: lo interpreta in simultanea senza generare il file. più lento ma funziona ovunque.

F2 = N1, Errore

```
int main() {
    int n1, n2, n3;
    float f1, f2, f3;
    n1 = 3;
    f2 = (double) n1;   ] → Casting
    f1 = 5.2;
    n2 = (int) f1;      ]
```

/* Per divisioni che int → */ f2 = 5/9 else

$f_2 = {}^{5.0}/_{9}$ o ${}^{5}/_{9.0}$ → converte

$^{int}/_{int}$

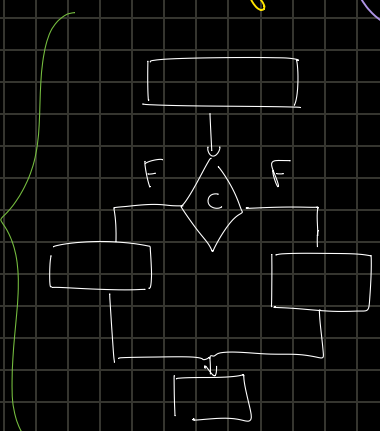
Gestire logica di un programma

Sarebbe lineare, ma esistono strutture che modificano il flusso:

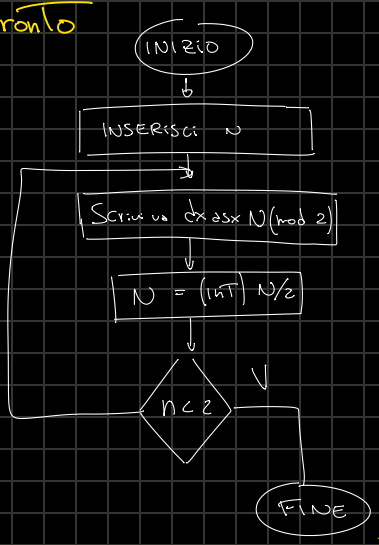
- Iterative: Do ... while, while ... do, for (...)
- Condizionali if (...) { ... } else { ... }

↳ operatori di confronto

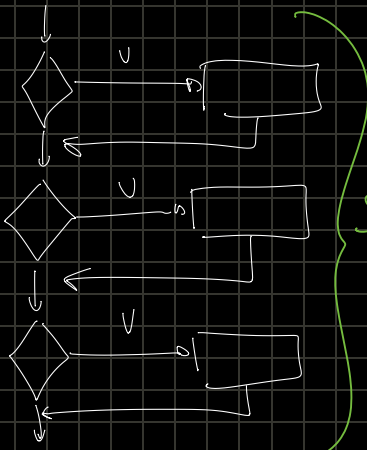
distruzione



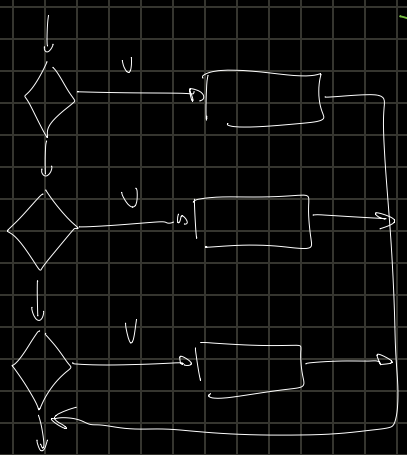
Struttura condizionale



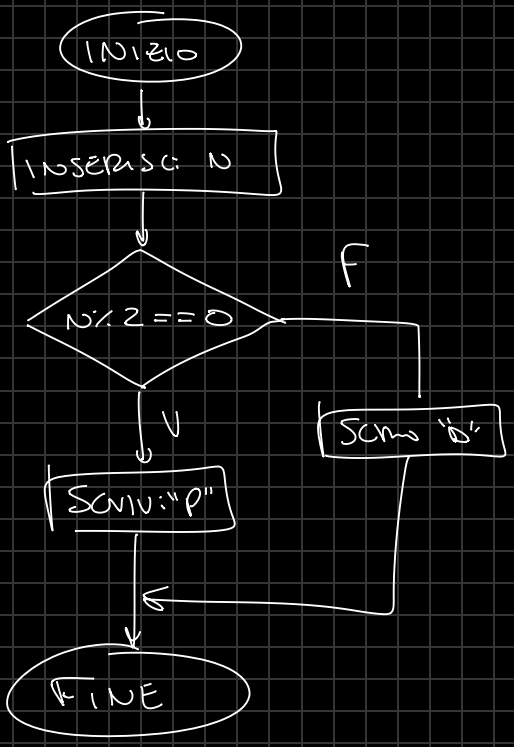
Struttura di iterazione



Struttura non esclusiva



esclusiva




```
#header

int main () {
    int n = 0;
    printf ("Inserisci un numero \n");
    scanf ("%i", &n);
    if ((n % 2) == 0) printf ("n e' pari \n");
    else printf ("n e' dispari");
}
```

```
do {
    Istruzioni;
    Vengono fatte
    quando la cond
    e' verificata
} while (cond)
```

```
while (cond) {
}
```

```
#header
int main () {
    int i = 0;
    do {
        i++;
        printf ("%i \n", i);
    } while (i <= 10);
    printf ("Fine! \n");
}
```

```
1
2
3
...
10
11
Fine!
```